

1

Властивості молекул

Після обчислень електронної структури молекули природним кроком є обчислення її спостережуваних властивостей: коливальних спектрів, оптичних переходів, електричних та магнітних характеристик. PySCF надає потужні інструменти для розрахунку цих властивостей на рівні теорії Хартрі-Фока та пост-ХФ методів.

У попередніх розділах ви навчилися виконувати основні квантово-хімічні розрахунки: оптимізувати геометрію, обчислювати енергії, аналізувати хвильові функції. Тепер настав час перейти до **обчислення молекулярних властивостей** — фізичних величин, які можна безпосередньо порівняти з експериментом.

Що таке молекулярні властивості?

Молекулярні властивості — це спостережувані величини, які характеризують поведінку молекули у зовнішніх полях або при взаємодії з випромінюванням. На відміну від енергії чи геометрії, властивості безпосередньо вимірюються експериментально:

- **Коливальні спектри (ІЧ, Раман):** частоти, інтенсивності.
- **Електронні спектри (УФ-видимі):** довжини хвиль, сили осциляторів.
- **Електричні властивості:** дипольний момент, поляризованість.
- **Магнітні властивості:** ЯМР зсуви, g-тензор, константи спин-спінової взаємодії.
- **Оптична активність:** круговий дихроїзм, оптичне обертання.

Головна перевага обчислення властивостей: можна безпосередньо порівняти теорію з експериментом та перевірити якість обчислень!

Встановлення додаткових модулів PySCF

Базова інсталяція PySCF містить функціонал для розрахунку енергій, градієнтів та оптимізації геометрії. Для обчислення спектроскопічних та магнітних властивостей потрібні **додаткові модулі**.

Основні модулі для властивостей

1. `pyscf-properties` — головний модуль для властивостей:

```
pip install pyscf-properties
```

Цей модуль включає:

- `pyscf.prop.nmr` — ЯМР константи екранування
- `pyscf.prop.esr` — ЕПР g-тензор, константи надтонкої взаємодії
- `pyscf.prop.polarizability` — поляризованість, гіперполяризованості
- `pyscf.prop.magnetizability` — магнітна сприйнятливості

2. `pyscf-geomopt` — для оптимізації геометрії:

```
pip install pyscf-geomopt
```

3. `geometric` — покращена оптимізація (рекомендується):

```
pip install geometric
```

Перевірка встановлення

Після інсталяції перевірте, чи працюють модулі:

```
import pyscf
from pyscf import gto, scf
from pyscf.prop import nmr, esr # для магнітних властивостей
from pyscf.hessian import thermo # для коливальних властивостей
from pyscf import tddft # для електронних збуджень

print("PySCF версія:", pyscf.__version__)
print("Модулі успішно імпортовані!")
```

Якщо імпорт проходить без помилок — все готово для роботи!

Альтернатива: встановлення з conda

Для користувачів Anaconda/Miniconda:

```
conda install -c pyscf pyscf
conda install -c conda-forge geometric
pip install pyscf-properties # properties поки немає в conda
```

Концептуальна основа: теорія відгуку

Теорія відгуку (response theory) є потужним інструментом для обчислення молекулярних властивостей у квантовій механіці. Її суть полягає в аналізі реакції системи на зовнішні збурення: ми застосовуємо слабе зовнішнє поле (або збурення) і спостерігаємо, як змінюється енергія, хвильова функція чи інші observable величини. Цей підхід дозволяє систематично обчислювати похідні енергії по параметрах збурення, які безпосередньо відповідають фізичним властивостям, таким як поляризованість, магнітні моменти чи спектроскопічні переходи.

Загалом, енергія системи в присутності збурення λ (де λ може бути будь-яким параметром, наприклад, електричним або магнітним полем чи геометрією молекули тощо) розкладається в ряд Тейлора:

$$E(\lambda) = E_0 + \left. \frac{\partial E}{\partial \lambda} \right|_{\lambda=0} \lambda + \frac{1}{2!} \left. \frac{\partial^2 E}{\partial \lambda^2} \right|_{\lambda=0} \lambda^2 + \frac{1}{3!} \left. \frac{\partial^3 E}{\partial \lambda^3} \right|_{\lambda=0} \lambda^3 + \dots$$

Коефіцієнти при степенях λ є похідними енергії по збуренню в точці $\lambda = 0$ і визначають лінійний, квадратичний тощо відгук системи. Ці похідні інтерпретуються як фундаментальні властивості:

- Перша похідна $\left. \frac{\partial E}{\partial \lambda} \right|_{\lambda=0}$ — лінійний відгук (наприклад, дипольний момент).
- Друга похідна $\left. \frac{\partial^2 E}{\partial \lambda^2} \right|_{\lambda=0}$ — квадратичний відгук (наприклад, поляризованість).
- Вищі похідні — нелінійні ефекти (гіперполяризованість тощо).

Загальний принцип: будь-яка молекулярна властивість, пов'язана з відгуком на збурення, є похідною енергії (або хвильової функції) по відповідному параметру.

Приклад: розклад енергії в електричному полі

Для ілюстрації розглянемо конкретний випадок статичного однорідного електричного поля $\mathbf{E}$. Енергія системи розкладається як:

$$E(\mathbf{E}) = E_0 - \mu \cdot \mathbf{E} - \frac{1}{2} \sum_{ij} \alpha_{ij} E_i E_j - \frac{1}{6} \sum_{ijk} \beta_{ijk} E_i E_j E_k + \dots$$

де:

- E_0 — енергія системи без поля.
- μ — статичний дипольний момент (перша похідна $-\left. \frac{\partial E}{\partial \mathbf{E}} \right|_{\mathbf{E}=0}$).
- α_{ij} — тензор поляризованості (друга похідна $-\left. \frac{\partial^2 E}{\partial E_i \partial E_j} \right|_{\mathbf{E}=0}$).
- β_{ijk} — тензор першої гіперполяризованості (третя похідна $-\left. \frac{\partial^3 E}{\partial E_i \partial E_j \partial E_k} \right|_{\mathbf{E}=0}$).

Цей розклад є частковим випадком загальної теорії відгуку, де $\lambda \equiv \mathbf{E}$. Аналогічно, теорія застосовується до магнітних полів (для магнітної сприйнятливості), геометричних змін (для сил і гессіанів) чи часозалежних збурень (для спектроскопії).

Загальний принцип:

властивість = похідна енергії по зовнішньому полю.

Методи обчислення похідних

1. Числові похідні (finite differences):

$$\alpha_{xx} = -\frac{\partial^2 E}{\partial E_x^2} \approx \frac{E(E_x) + E(-E_x) - 2E(0)}{\Delta E_x^2}$$

Плюси: просто, універсально

Мінуси: неточно, повільно, багато розрахунків

2. Аналітичні похідні:

$$\alpha_{ij} = -\left. \frac{\partial^2 E}{\partial E_i \partial E_j} \right|_{E=0} = \text{складна формула через хвильову функцію}$$

Плюси: точно, швидко, один розрахунок

Мінуси: складна реалізація

3. Coupled-Perturbed методи (CP-HF, CP-KS):

Розв'язують рівняння для змін орбіталей під дією поля. Використовується в PySCF для більшості властивостей.

Рекомендація: завжди використовуйте аналітичні похідні, коли вони доступні!

Gauge-invariance для магнітних властивостей

Магнітні властивості (ЯМР, магнітна сприйнятливність) мають специфічну проблему: результат залежить від вибору **gauge** (калібрування) векторного потенціалу.

Проблема gauge origin

Магнітне поле $\mathbf{B}$ можна записати через векторний потенціал:

$$\mathbf{B} = \nabla \times \mathbf{A}$$

Але $\mathbf{A}$ не унікальний! Калібрувальне перетворення $\mathbf{A} \rightarrow \mathbf{A} + \nabla \chi$ не змінює $\mathbf{B}$.

Для скінченного базису результат залежить від вибору початку координат для $\mathbf{A}$ — це нефізично!

Рішення: GIAO (Gauge-Independent Atomic Orbitals)

У PySCF використовується метод GIAO (також називається IGLO, CSGT):

- Кожна атомна орбіталь має своє локальне калібрування.
- Результати не залежать від вибору початку координат.
- Швидка збіжність з розміром базису.
- **Стандарт** для розрахунків ЯМР властивостей.

В PySCF це працює автоматично — не потрібно нічого додатково налаштовувати!

```
from pyscf.prop import nmr

# GIAO використовується автоматично
nmr_calc = nmr.RHF(mf)
shielding = nmr_calc.kernel() # gauge-independent результат!
```

Базиси для розрахунку властивостей

Важливо: базиси для енергій $\neq$ базиси для властивостей!

Загальні вимоги

1. Дифузні функції (aug-, d-aug-):

Критично важливі для:

- Поляризовностей (опис деформації електронної густини)
- Магнітних властивостей (струми далеко від ядер)
- Анізотропних властивостей

Базиси: aug-cc-pVDZ, aug-cc-pVTZ, 6-311++G**

2. Високий angular momentum (поляризаційні функції):

Для точних властивостей потрібні d , f , іноді g функції.

3. Tight functions для ЯМР:

ЯМР екранування чутливе до густини на ядрі $\rho(\mathbf{R}_A)$. Спеціальні базиси: pcS-n, pcJ-n

Спеціалізовані базиси

Властивість	Рекомендований базис	Коментар
ІЧ частоти	6-31G*, 6-311G**	Невеликі базиси ОК
УФ-видимі спектри	aug-cc-pVDZ/TZ	Дифузні важливі
Поляризованість	aug-cc-pVDZ	Обов'язково aug-
ЯМР екранування	pcS-2, aug-cc-pVTZ	Tight + diffuse
J-константи	pcJ-2, pcJ-3	Спеціальний базис
ЕПР (g-тензор, HFC)	EPR-II, EPR-III	Tight s-функції

Практичні поради

- Для початку: **aug-cc-pVDZ** — універсальний вибір
- Для точних результатів: **aug-cc-pVTZ**
- Для великих систем: **6-311+G(2d,p)** — компроміс
- Для ЯМР: спеціалізовані базиси **pcS-n** кращі за aug-cc-pVnZ

Структура розділу

Розділ організовано за типами властивостей:

1. **Коливальні властивості (Секція 1.1):**
 - Гессіан та нормальні моди
 - ІЧ спектри (інтенсивності, правила відбору)
 - Раман спектри
 - Термохімія та ZPE
2. **Електронні властивості (Секція ??):**
 - Електронні збудження (TD-DFT)
 - УФ-видимі спектри
 - Флуоресценція та фосфоресценція
3. **Електричні властивості (Секція ??):**
 - Дипольний момент
 - Поляризованість (статична та динамічна)
 - Гіперполяризованості
 - Електростатичний потенціал
4. **Магнітні властивості (Секція ??):**
 - Магнітна сприйнятливості
 - ЯМР константи екранування та J-константи
 - ЕПР властивості (g-тензор, HFC)
 - Оптична активність (ORD, CD)

Практичні рекомендації перед початком

Типовий workflow розрахунку властивостей

1. Оптимізуйте геометрію:

```
from pyscf import gto, scf
from pyscf.geomopt.geometric_solver import optimize

mol = gto.M(atom='...', basis='6-31g*')
mf = scf.RHF(mol).run()
mol_eq = optimize(mf) # оптимізована геометрія
```

2. Перерахуйте з кращим базисом:

```
mol_prop = gto.M(atom=mol_eq.atom, basis='aug-cc-pvdz')
mf_prop = scf.RHF(mol_prop).run()
```

3. Обчисліть властивість:

```
from pyscf.prop import nmr
nmr_calc = nmr.RHF(mf_prop)
shielding = nmr_calc.kernel()
```

4. Проаналізуйте результат!

Типові помилки початківців

- Розрахунок властивостей без оптимізації геометрії
- Використання малих базисів (6-31G) для поляризовностей
- Забування про дифузні функції для анізотропних властивостей
- Розрахунок ЯМР без gauge-independent методу
- Порівняння абсолютних значень замість зсувів (для ЯМР)

Контрольний чек-лист

Перед розрахунком властивості перевірте:

- Геометрія оптимізована? (немає уявних частот)
- Базис підходить для цієї властивості?
- Метод підходить? (HF завищує поляризованість, потрібен DFT/MP2)
- Для магнітних: gauge-independent метод?
- SCF збігся? (check `mf.converged`)
- Для радикалів: перевірили $\langle S^2 \rangle$?

Корисні ресурси

- Документація PySCF: <https://pyscf.org/user.html>
- PySCF properties: <https://github.com/pyscf/properties>
- Basis Set Exchange: <https://www.basissetexchange.org/>
- NIST WebBook: експериментальні дані для порівняння

1.1. Коливальні властивості молекул

Коливальна спектроскопія — один із найпотужніших методів ідентифікації молекул та вивчення їх структури. Інфрачервоні (ІЧ) та Раман спектри надають інформацію про коливальні моди, силові константи зв'язків, та симетрію молекули.

Частоти в різних одиницях:

Одиниця	Множник	Використання
см ⁻¹	1	ІЧ спектроскопія
Гц	2.998×10^{10}	СІ одиниці
еВ	1.240×10^{-4}	Електронна спектроскопія
Хартрі	4.556×10^{-6}	Атомні одиниці

1.1.1. Гесіан та нормальні моди

Коливання атомів у молекулі відбуваються поблизу рівноважної геометрії $\mathbf{R}_0$. Для малих відхилень потенційна енергія системи можна апроксимувати розкладом у ряд Тейлора до другого порядку:

$$E(\mathbf{R}) \approx E(\mathbf{R}_0) + \frac{1}{2} \sum_{ij} H_{ij} \Delta R_i \Delta R_j,$$

де

$$H_{ij} = \left. \frac{\partial^2 E}{\partial R_i \partial R_j} \right|_{\mathbf{R}_0}$$

— елементи *гесіану* (матриці других похідних енергії за декартовими координатами ядер).

Гесіан має розмірність $3N \times 3N$ для молекули з N атомів і містить повну інформацію про гармонічні коливальні властивості системи.

Матриця силових констант

Гесіан також називають **матрицею силових констант**, оскільки його елементи відображають реакцію потенціалу на малі зміщення ядер:

$$H_{ij} = \left. \frac{\partial^2 E}{\partial R_i \partial R_j} \right|_{\mathbf{R}_0}$$

Фізичний зміст елементів гесіану:

- *Діагональні елементи* — характеризують жорсткість потенціалу вздовж окремих координат;
- *Недіагональні елементи* — описують зв'язок між зміщеннями різних атомів;
- *Власні значення* ω^2 — квадрати коливальних частот;
- *Власні вектори* — напрямки нормальних мод.

Таким чином, діагоналізація гесіану дозволяє отримати нормальні координати та спектр гармонічних частот, що повністю характеризують коливальну поведінку молекули поблизу рівноваги.

Нормальні коливання

Для переходу від гесіану $\mathbf{H}$ у декартових координатах до фізично осмислених частот враховують маси атомів. Вводять **мас-зважений гесіан**:

$$\mathbf{F} = \mathbf{M}^{-1/2} \mathbf{H} \mathbf{M}^{-1/2},$$

де $\mathbf{M}$ — діагональна матриця атомних мас.

Далі розв'язують задачу на власні значення:

$$\mathbf{F} \mathbf{Q} = \omega^2 \mathbf{Q},$$

де ω — коливальні частоти, а $\mathbf{Q}$ — вектори нормальних координат.

Отримані власні значення ω_i^2 відповідають:

- $3N - 6$ коливальним модам для нелінійної молекули;
- $3N - 5$ — для лінійної;
- 3 трансляційним і 3 (або 2) обертальним модам з $\omega = 0$.

Зв'язок координат Q_k та $\mathbf{R}$ Нормальні координати Q_k є лінійними комбінаціями атомних зміщень $\Delta \mathbf{R}$:

$$Q_k = \sum_{i=1}^{3N} \sqrt{m_i} l_{ik} \Delta R_i$$

або у матричній формі:

$$\mathbf{Q} = \mathbf{L}^T \mathbf{M}^{1/2} \Delta \mathbf{R},$$

де:

- $\mathbf{M}$ — діагональна матриця мас атомів,
- $\mathbf{L}$ — матриця власних векторів (нормальних мод),
- $\Delta \mathbf{R} = \mathbf{R} - \mathbf{R}_0$ — відхилення координат від рівноваги.

Звідси обернене перетворення:

$$\Delta \mathbf{R} = \mathbf{M}^{-1/2} \mathbf{L} \mathbf{Q}_k.$$

Саме цю формулу використовують у чисельному диференціюванні:

$$\mathbf{R}^{(\pm)} = \mathbf{R}_0 \pm h \mathbf{M}^{-1/2} \mathbf{L}_k,$$

що зв'язує крок h у просторі нормальних координат Q_k із фізичними зміщеннями атомів $\mathbf{R}$.

Масштабування частот

Розраховані гармонічні частоти $\nu_{\text{гарм}}$ систематично завищені відносно експериментальних $\nu_{\text{експ}}$ через спрощення моделі потенціальної поверхні та обмеження методів:

- **Гармонічне наближення:** реальний потенціал ангармонічний, тому енергія коливань і частоти менші.
- **Нестача електронної кореляції:** метод Хартрі–Фока завищує жорсткість зв'язків.
- **Обмежений базис:** малий базис перебільшує градієнти енергії.

Масштабування λ емпірично компенсує ангармонічність і методичні похибки. Емпіричне виправлення:

$$\nu_{\text{корект}} = \lambda \cdot \nu_{\text{гарм}}$$

Типові масштабувальні фактори:

Метод/базис	λ	Коментар
HF/6-31G*	0.8929	Сильно завищує
B3LYP/6-31G*	0.9613	Універсальний
B3LYP/6-311+G(2d,p)	0.9679	Кращий базис
MP2/6-31G*	0.9434	Для точних розрахунків

Джерело: NIST Computational Chemistry Comparison and Benchmark Database

1.1.2. Обчислення гесіану

PySCF може обчислювати гесіан аналітично або чисельно:

`h2o_hessian.py`

```
# =====
# h2o_hessian.py
# Обчислення гесіану чисельним та аналітичним способами
# Їх порівняння
# =====

import time

import numpy as np
from pyscf import gto, scf
from pyscf.hessian import rhf as rhf_hess

# Створення молекули води
mol = gto.M(
    atom="""
O  0.0000  0.0000  0.1173
H  0.0000  0.7572 -0.4692
H  0.0000 -0.7572 -0.4692
""",
    basis="6-31g",
    unit="angstrom",
)

print("=" * 60)
print("Обчислення Гесіану для молекули води (H2O)")
print("=" * 60)
print(f"\nБазис: {mol.basis}")
```

```
print(f"Кількість атомів: {mol.natm}")
print(f"Розмір Гесіану: {3 * mol.natm} x {3 * mol.natm}")

# Виконання SCF розрахунку
print("\n--- Розрахунок SCF ---")
mf = scf.RHF(mol)
mf.kernel()
print(f"Енергія SCF: {mf.e_tot:.10f} Ha")

# =====
# 1. ЧИСЕЛЬНИЙ ГЕСІАН (числове диференціювання градієнта)
# =====
print("\n" + "=" * 60)
print("1. ЧИСЕЛЬНИЙ ГЕСІАН")
print("=" * 60)
print("Обчислюємо через скінченні різниці градієнтів...")

start_time = time.time()

# Крок для числового диференціювання
step = 0.001 # в атомних одиницях (Bohr)

natm = mol.natm
hess_numerical = np.zeros((natm, natm, 3, 3))

# Обчислюємо градієнт для кожного зміщення
for atom_i in range(natm):
    for coord_i in range(3):
        # Позитивне зміщення
        coords_pos = mol.atom_coords().copy()
        coords_pos[atom_i, coord_i] += step

        mol_pos = gto.M(
            atom=[mol.atom_symbol(i), coords_pos[i]] for i in range(natm)],
            basis=mol.basis,
            unit="Bohr",
        )
        mf_pos = scf.RHF(mol_pos)
        mf_pos.verbose = 0
        mf_pos.kernel()
        grad_pos = mf_pos.Gradients().kernel()

        # Негативне зміщення
        coords_neg = mol.atom_coords().copy()
        coords_neg[atom_i, coord_i] -= step

        mol_neg = gto.M(
            atom=[mol.atom_symbol(i), coords_neg[i]] for i in range(natm)],
            basis=mol.basis,
            unit="Bohr",
        )
        mf_neg = scf.RHF(mol_neg)
        mf_neg.verbose = 0
        mf_neg.kernel()
        grad_neg = mf_neg.Gradients().kernel()

        # Числова похідна:  $d^2E/dx_i dx_j = (dE/dx_j|_{x_i+h} - dE/dx_j|_{x_i-h}) / 2h$ 
        hess_numerical[:, atom_i, :, coord_i] = (grad_pos - grad_neg) / (2 * step)

numerical_time = time.time() - start_time

print(f"\nЧас обчислення: {numerical_time:.2f} c")
print(f"Форма Гесіану: {hess_numerical.shape}")
```

```

# Перетворюємо в 2D для виводу
hess_num_2d = hess_numerical.transpose(0, 2, 1, 3).reshape(3 * natm, 3 * natm)
print("\nЧисельний Гесіан (перші 6x6 елементів):")
print(hess_num_2d[:6, :6])

# =====
# 2. АНАЛІТИЧНИЙ ГЕСІАН (через аналітичні похідні)
# =====
print("\n" + "=" * 60)
print("2. АНАЛІТИЧНИЙ ГЕСІАН")
print("=" * 60)

start_time = time.time()

# Аналітичний Гесіан через pyscf.hessian.rhf
hess_analytical = rhf_hess.Hessian(mf).kernel()

analytical_time = time.time() - start_time

print(f"\nЧас обчислення: {analytical_time:.2f} с")
print(f"Форма Гесіану: {hess_analytical.shape}")

# Перетворюємо в 2D для виводу
hess_anal_2d = hess_analytical.transpose(0, 2, 1, 3).reshape(3 * natm, 3 * natm)
print("\nАналітичний Гесіан (перші 6x6 елементів):")
print(hess_anal_2d[:6, :6])

# =====
# ПОРІВНЯННЯ РЕЗУЛЬТАТІВ
# =====
print("\n" + "=" * 60)
print("ПОРІВНЯННЯ РЕЗУЛЬТАТІВ")
print("=" * 60)

difference = hess_num_2d - hess_anal_2d
max_diff = np.max(np.abs(difference))
mean_diff = np.mean(np.abs(difference))

print(f"\nМаксимальна різниця: {max_diff:.2e}")
print(f"Середня абсолютна різниця: {mean_diff:.2e}")
print(f"\nПрискорення (аналітичний швидший у): {numerical_time / analytical_time:.2f}x")

print("\nМатриця різниць (перші 6x6 елементів):")
print(difference[:6, :6])

# =====
# ВЛАСНІ ЗНАЧЕННЯ (частоти коливань)
# =====
print("\n" + "=" * 60)
print("ВЛАСНІ ЗНАЧЕННЯ ГЕСІАНУ (з аналітичного)")
print("=" * 60)

# Перетворюємо Гесіан з форми (natm, natm, 3, 3) в (3*natm, 3*natm)
hess_2d = hess_analytical.transpose(0, 2, 1, 3).reshape(3 * natm, 3 * natm)

# Масово-зважений Гесіан для частот
mass = np.array([mol.atom_mass_list()[i] for i in range(natm)])
mass_matrix = np.repeat(mass, 3)
mass_weighted_hess = hess_2d / np.sqrt(mass_matrix[:, None] * mass_matrix[None, :])

eigenvalues, eigenvectors = np.linalg.eigh(mass_weighted_hess)

# Конвертація в частоти (cm^-1)
# 1 Hartree/(amu*Bohr^2) * (a0/Bohr)^2 = omega^2

```

```
# omega = sqrt(eigenvalue) * conversion_factor
conversion = 5140.48 # приблизний коефіцієнт для Ha/(amu*angstrom^2) -> cm^-1

frequencies = np.sign(eigenvalues) * np.sqrt(np.abs(eigenvalues)) * conversion

print("\nЧастоти коливань (см^-1):")
print("(перші 6 значень мають бути близькі до нуля - трансляції та обертання)")
for i, freq in enumerate(frequencies):
    mode_type = "трансляція/обертання" if i < 6 else "коливання"
    print(f"  Мода {i + 1}: {freq:10.2f} см^-1 ({mode_type})")

print("\n" + "=" * 60)
print("ПРИМІТКИ:")
print("=" * 60)
print("1. Чисельний метод: диференціювання градієнтів методом центральних різниць")
print("2. Аналітичний метод: pyscf.hessian.rhf.Hessian - точні другі похідні")
print("3. Аналітичний метод набагато швидший та точніший!")
print("=" * 60)
```

Програма обчислює матрицю Гесіану (матрицю других похідних енергії) для молекули води (H_2O) двома методами: чисельним та аналітичним.

Коментар

У програмному пакеті PySCF гесіан зберігається не як звичайна двовимірна матриця розмірності $3N \times 3N$, а у вигляді чотиривимірного тензора

$$H_{AB,ij} = \frac{\partial^2 E}{\partial R_{A,i} \partial R_{B,j}},$$

де A, B — індекси атомів ($1, \dots, N$), а $i, j \in \{x, y, z\}$ — напрямки декартових координат. Відповідно, внутрішня структура масиву має вигляд

$$H \in \mathbb{R}^{N \times N \times 3 \times 3}.$$

Щоб отримати стандартну матричну форму $3N \times 3N$, необхідно спершу переставити осі, розмістивши напрямки координат поруч із відповідними атомами:

$$H'_{Ai,Bj} = H_{AB,ij}.$$

Це виконується операцією

$$H' = H.\text{transpose}(0, 2, 1, 3).\text{reshape}(3N, 3N).$$

Така трансформація формує коректну блочну структуру з блоками 3×3 , яка відповідає фізичному гесіану молекули у декартових координатах.

Чисельний Гесіан

Чисельний Гесіан обчислюється через скінченні різниці методом центральних різниць:

1. **Зміщення координат:** Кожна координата x_i кожного атома зміщується на величину $\pm h$ (крок диференціювання):

$$x_i^{(\pm)} = x_i \pm h \quad (1.1)$$

де $h = 0.001$ Bohr (атомні одиниці).

2. **Обчислення градієнтів:** Для кожного зміщення обчислюється градієнт енергії:

$$\nabla E(x_i^{(+)}) = \left(\frac{\partial E}{\partial x_1}, \frac{\partial E}{\partial x_2}, \dots, \frac{\partial E}{\partial x_N} \right)_{x_i=x_i+h} \quad (1.2)$$

$$\nabla E(x_i^{(-)}) = \left(\frac{\partial E}{\partial x_1}, \frac{\partial E}{\partial x_2}, \dots, \frac{\partial E}{\partial x_N} \right)_{x_i=x_i-h} \quad (1.3)$$

3. **Обчислення Гесіану:** Елементи матриці Гесіану обчислюються як:

$$H_{ij} = \frac{\partial^2 E}{\partial x_i \partial x_j} = \frac{\nabla E_j(x_i^{(+)}) - \nabla E_j(x_i^{(-)})}{2h} \quad (1.4)$$

Формула центральних різниць:

$$H_{ij} = \frac{g_j(x_i + h) - g_j(x_i - h)}{2h} \quad (1.5)$$

де $g_j = \frac{\partial E}{\partial x_j}$ — компонента градієнта.

Аналітичний Гесіан

Аналітичний Гесіан обчислюється через точні другі похідні енергії, використовуючи модуль PySCF:

$$H_{ij} = \frac{\partial^2 E}{\partial x_i \partial x_j} \quad (1.6)$$

Метод використовує `pyscf.hessian.rhf.Hessian(mf).kernel()` для прямого обчислення всіх елементів матриці Гесіану без числового диференціювання.

Обчислювальна складність

- **Чисельний метод:** Потребує $2 \times 3N$ SCF розрахунків, де N — кількість атомів.
 - Для H_2O ($N = 3$): $2 \times 3 \times 3 = 18$ SCF розрахунків.
 - Час виконання: $\sim 10 - 30$ секунд.
- **Аналітичний метод:** Один розрахунок з аналітичними похідними.
 - Час виконання: $\sim 1 - 5$ секунд.
 - **Прискорення:** у $5 - 20$ разів швидше.

Точність

- **Аналітичний метод:** Точніший, оскільки використовує точні похідні без похибок дискретизації

- **Чисельний метод:** Точність залежить від кроку h :

$$\text{Похибка} \sim O(h^2) + \frac{\varepsilon}{h} \quad (1.7)$$

де ε — машинна точність

- **Типова різниця:** Максимальна абсолютна різниця $\sim 10^{-5}$ – 10^{-7} Ha/Bohr².

Коливальні частоти

З матриці Гесіану можна отримати коливальні частоти молекули:

$$\omega_i = \sqrt{\lambda_i} \times 5140.48, \quad (1.8)$$

де 5140.48 — це числовий множник, який переводить частоти з атомних одиниць у cm^{-1} , а λ_i — власні значення масово-зваженого Гесіану:

$$\tilde{H}_{ij} = \frac{H_{ij}}{\sqrt{m_i m_j}} \quad (1.9)$$

Очікувані частоти для H_2O :

- Перші 6 мод: $\approx 0 \text{ cm}^{-1}$ (трансляції та обертання)
- Симетричне валентне коливання: $\approx 3600 \text{ cm}^{-1}$
- Деформаційне коливання: $\approx 1600 \text{ cm}^{-1}$
- Асиметричне валентне коливання: $\approx 3800 \text{ cm}^{-1}$

Для практичних розрахунків **аналітичний метод є кращим вибором**, оскільки він значно швидший та точніший. Чисельний метод корисний для перевірки результатів або коли аналітичні похідні недоступні.

1.1.3. Інфрачервоні (ІЧ) спектри

ІЧ-спектроскопія досліджує взаємодію молекул з інфрачервоним випромінюванням. Поглинання відбувається тоді, коли частота випромінювання збігається з частотою внутрішнього коливання молекули, тобто виконується умова резонансу:

$$h\nu = \Delta E_{\text{колив.}}$$

Ці переходи відповідають змінам *коливальних* рівнів енергії. Типові частоти для молекулярних коливань лежать у межах

$$400 - 4000 \text{ cm}^{-1},$$

що відповідає довжинам хвиль у мікрометровому діапазоні ($2.5 - 25 \mu\text{m}$).

ІЧ-спектр відображає лише ті *коливальні моди молекули*, які супроводжуються *зміною її дипольного моменту*.

Інтенсивності ІЧ поглинання

ІЧ інтенсивність пропорційна квадрату зміни дипольного моменту:

$$I_i \propto \left| \frac{d\mu}{dQ_i} \right|^2$$

де Q_i — нормальна координата i -ї моди, μ — дипольний момент.

Правило відбору:

Коливання є **ІЧ-активним** тільки якщо:

$$\left| \frac{\partial \mu}{\partial Q_k} \right| \neq 0 \quad (1.10)$$

Це означає, що дипольний момент молекули має змінюватися під час коливання. Симетрична молекула може мати *ІЧ-неактивні* моди.

Повний вектор похідної дипольного моменту по нормальним координатам Q_k :

$$\frac{\partial \mu}{\partial Q_k} = \left(\frac{\partial \mu_x}{\partial Q_k}, \frac{\partial \mu_y}{\partial Q_k}, \frac{\partial \mu_z}{\partial Q_k} \right) \quad (1.11)$$

Його норма:

$$\left| \frac{\partial \mu}{\partial Q_k} \right| = \sqrt{\left(\frac{\partial \mu_x}{\partial Q_k} \right)^2 + \left(\frac{\partial \mu_y}{\partial Q_k} \right)^2 + \left(\frac{\partial \mu_z}{\partial Q_k} \right)^2} \quad (1.12)$$

Інтенсивність обчислюється як:

$$I_k = K \times \left| \frac{\partial \mu}{\partial Q_k} \right|^2, \quad (1.13)$$

Дипольний момент в атомних одиницях (a.u. = $e \cdot a_0$):

$$1 \text{ a.u.} = 2.5418 \text{ Debye} \quad (1.14)$$

де e — заряд електрона, $a_0 = 0.529177 \text{ \AA}$ — радіус Бора.

Похідна в атомних одиницях має розмірність:

$$\left[\frac{\partial \mu}{\partial Q} \right] = \frac{e \cdot a_0}{a_0 \cdot \sqrt{\text{amu}}} = \frac{e}{\sqrt{\text{amu}}} \quad (1.15)$$

Отже:

$$I \text{ (кМ/моль)} = \frac{N_A \pi}{3c^2} \left| \frac{\partial \mu}{\partial Q} \right|^2 \quad (1.16)$$

де:

- $N_A = 6.022 \times 10^{23} \text{ моль}^{-1}$ — число Авогадро
- $c = 2.998 \times 10^{10} \text{ см/с}$ — швидкість світла

Після підстановки констант:

$$I = 974.8638 \times \left| \frac{\partial \mu}{\partial Q} \right|_{\text{a.u.}}^2 \quad (1.17)$$

1.1.4. Класифікація за інтенсивністю

Інтенсивність (кМ/моль)	Класифікація
$I > 100$	Сильна смуга
$10 < I \leq 100$	Середня смуга
$I \leq 10$	Слабка смуга

1.1.5. ІЧ спектр H_2O

Програма використовує **метод центральних різниць** для чисельного обчислення $\frac{\partial \mu}{\partial Q_k}$:

$$\frac{\partial \mu}{\partial Q_k} \approx \frac{\mu(Q_k + h) - \mu(Q_k - h)}{2h} \quad (1.18)$$

де $h = 0.001$ Bohr — крок диференціювання.

Алгоритм обчислення

1. Отримання нормальної моди:

З діагоналізації масово-зваженого Гесіану отримуємо власний вектор $\mathbf{L}_k$ (нормальну моду):

$$\mathbf{L}_k = (l_{1x}, l_{1y}, l_{1z}, l_{2x}, l_{2y}, l_{2z}, \dots, l_{Nx}, l_{Ny}, l_{Nz})^T \quad (1.19)$$

де N — кількість атомів.

2. Зміщення координат:

Зміщуємо координати всіх атомів вздовж нормальної моди:

$$\mathbf{R}^{(+)} = \mathbf{R}_0 + h \cdot \mathbf{L}_k \quad (1.20)$$

$$\mathbf{R}^{(-)} = \mathbf{R}_0 - h \cdot \mathbf{L}_k \quad (1.21)$$

3. SCF розрахунки:

Для кожної зміщеної геометрії виконуємо SCF розрахунок:

$$E^{(+)}, \mu^{(+)} = \text{SCF}(\mathbf{R}^{(+)}) \quad (1.22)$$

$$E^{(-)}, \mu^{(-)} = \text{SCF}(\mathbf{R}^{(-)}) \quad (1.23)$$

4. Обчислення похідної:

Чисельна похідна дипольного моменту:

$$\frac{\partial \mu}{\partial Q_k} = \frac{\mu^{(+)} - \mu^{(-)}}{2h} \quad (1.24)$$

h2o_ir_spectrum.py

```

# =====
# h2o_ir_spectrum.py
# Розрахунок ІЧ-спектру (частоти + інтенсивності)
# =====

import numpy as np
from pyscf import gto, scf
from pyscf.hessian import rhf as rhf_hess
from pyscf.hessian import thermo
import matplotlib.pyplot as plt

# =====
# Створення молекули H2O
# =====
mol = gto.M(
    atom="""
    O  0.0000  0.0000  0.1173
    H  0.0000  0.7572 -0.4692
    H  0.0000 -0.7572 -0.4692
    """,
    basis="6-31g",
    unit="angstrom",
)

print("=" * 70)
print("Розрахунок ІЧ-спектру H2O (частоти + інтенсивності)")
print("=" * 70)

# =====
# SCF розрахунок
# =====
print("\n[1/4] SCF розрахунок...")
mf = scf.RHF(mol)
mf.kernel()
print(f"Енергія: {mf.e_tot:.8f} Ha")

# =====
# Обчислення Гесіану
# =====
print("\n[2/4] Обчислення Гесіану...")
hess = mf.Hessian().kernel()

# =====
# Аналіз нормальних мод
# =====
print("\n[3/4] Аналіз нормальних мод...")
freq_info = thermo.harmonic_analysis(mol, hess)

frequencies = freq_info['freq_wavenumber'] # в см-1
normal_modes = freq_info['norm_mode'] # власні вектори

# =====
# Обчислення інтенсивностей ІЧ
# =====
print("\n[4/4] Обчислення інтенсивностей ІЧ...")

# Крок для числового диференціювання дипольного моменту
step = 0.001 # Bohr

natm = mol.natm
intensities = []

for mode_idx in range(len(frequencies)):

```

```

# Нормальна мода (3*natm вектор)
mode = normal_modes[:, mode_idx].reshape(natm, 3)

# Зміщення вздовж нормальної моди
coords_orig = mol.atom_coords() # в Bohr

# Позитивне зміщення
coords_pos = coords_orig + step * mode
mol_pos = gto.M(
    atom=[[mol.atom_symbol(i), coords_pos[i]] for i in range(natm)],
    basis=mol.basis,
    unit='Bohr'
)
mf_pos = scf.RHF(mol_pos)
mf_pos.verbose = 0
mf_pos.kernel()
dip_pos = mf_pos.dip_moment(unit='AU') # атомні одиниці

# Негативне зміщення
coords_neg = coords_orig - step * mode
mol_neg = gto.M(
    atom=[[mol.atom_symbol(i), coords_neg[i]] for i in range(natm)],
    basis=mol.basis,
    unit='Bohr'
)
mf_neg = scf.RHF(mol_neg)
mf_neg.verbose = 0
mf_neg.kernel()
dip_neg = mf_neg.dip_moment(unit='AU')

# Похідна дипольного моменту: dμ/dQ
dip_derivative = (dip_pos - dip_neg) / (2 * step)

# Інтенсивність: I ∝ |dμ/dQ|^2
# В одиницях: (Debye/Angstrom)^2 -> km/mol
# Константа: 974.8638 для конвертації
intensity = np.linalg.norm(dip_derivative)**2 * 974.8638
intensities.append(intensity)

intensities = np.array(intensities)

# =====
# Виведення результатів
# =====
print("\n" + "=" * 70)
print("ІЧ-СПЕКТР H2O")
print("=" * 70)
print(f"{'Мода':<8} {'ν (см⁻¹)':<15} {'Інтенсивність (km/mol)':<25} {'Тип'}")
print("-" * 70)

for i, (freq, intens) in enumerate(zip(frequencies, intensities)):
    if freq > 100: # Тільки справжні коливання (не трансляції/обертання)
        mode_type = "сильна" if intens > 100 else ("середня" if intens > 10 else "слабка")
        print(f"{i+1:<8} {freq:>12.1f} {intens:>20.1f} {mode_type}")

# =====
# Візуалізація спектру
# =====
print("\n[Візуалізація] Побудова ІЧ-спектру...")

# Фільтруємо тільки справжні коливання
real_modes = frequencies > 100
freq_plot = frequencies[real_modes]
intens_plot = intensities[real_modes]

```

```

# Створюємо спектр з гауссовим розширенням
x = np.linspace(0, 4500, 2000)
y = np.zeros_like(x)
fwhm = 20 # Full Width at Half Maximum (cm-1)

for freq, intens in zip(freq_plot, intens_plot):
    sigma = fwhm / (2 * np.sqrt(2 * np.log(2)))
    y += intens * np.exp(-((x - freq)**2) / (2 * sigma**2))

# Побудова графіку
plt.figure(figsize=(12, 6))
plt.plot(x, y, 'b-', linewidth=1.5)
plt.fill_between(x, y, alpha=0.3)

# Додаємо вертикальні лінії для піків
for freq, intens in zip(freq_plot, intens_plot):
    plt.axvline(freq, color='r', linestyle='--', alpha=0.5, linewidth=0.8)
    plt.text(freq, intens * 1.05, f'{freq:.0f}',
             rotation=90, va='bottom', ha='right', fontsize=8)

plt.xlabel('Хвильове число (см-1)', fontsize=12)
plt.ylabel('Інтенсивність (km/mol)', fontsize=12)
plt.title('ІЧ-спектр H2O (RHF/6-31G)', fontsize=14, fontweight='bold')
plt.grid(True, alpha=0.3)
plt.xlim(0, 4500)
plt.ylim(0, max(y) * 1.2)

plt.tight_layout()
plt.savefig('h2o_ir_spectrum.png', dpi=300, bbox_inches='tight')
print("Спектр збережено в 'h2o_ir_spectrum.png'")
plt.show()

# =====
# Інтерпретація мод
# =====
print("\n" + "=" * 70)
print("ІНТЕРПРЕТАЦІЯ КОЛИВАЛЬНИХ МОД")
print("=" * 70)
print("""
Для H2O очікуються 3 коливальні моди:

1. Симетричне валентне (ν1): ~3600 см-1
   - Обидва H-O зв'язки розтягуються синхронно
   - Середня інтенсивність

2. Деформаційне (ν2): ~1600 см-1
   - Зміна кута H-O-H
   - Найсильніша інтенсивність

3. Асиметричне валентне (ν3): ~3800 см-1
   - Один H-O зв'язок розтягується, інший стискається
   - Сильна інтенсивність

Примітка: Абсолютні значення можуть відрізнятися від експерименту
через наближення (базис 6-31G, RHF метод).
""")

print("=" * 70)
print("Розрахунок завершено!")
print("=" * 70)

```

1.1.6. Результати для H₂O

Нормальні моди води

Молекула H₂O має $3N - 6 = 3$ коливальні моди:

Мода	ν (см ⁻¹)	I (км/моль)	Тип
1	1828.9	1.4	Слабка (деформаційна)
2	3906.0	102.3	Сильна (симетричне валентне)
3	4001.1	138.6	Сильна (асиметричне валентне)

Примітка. Усі три моди молекули H₂O є ІЧ-активними (група симетрії C_{2v}). Найсильніше поглинання відповідає валентним коливанням, що зумовлюють інтенсивну ІЧ-смугу пари води в атмосфері (важливий внесок у парниковий ефект).

Обчислювальна складність

Для молекули з M коливальними модами (без трансляцій/обертань):

$$N_{\text{SCF}} = 1 + 2M \quad (1.25)$$

- 1 — початковий SCF для рівноважної геометрії
- $2M$ — по два SCF ($\pm h$) для кожної моди

Для H₂O: $N_{\text{SCF}} = 1 + 2 \times 3 = 7$ розрахунків.

Типовий час для H₂O/6-31G:

- Гесіан: ~ 1 – 2 с
- ІЧ інтенсивності: ~ 5 – 10 с
- Загальний час: ~ 6 – 12 с

Чисельні похибки

1. Вибір кроку h :

- Занадто великий h — похибка дискретизації $O(h^2)$
- Занадто малий h — похибка округлення $\sim \epsilon/h$
- Оптимальний $h \sim 10^{-3}$ Bohr

2. Збіжність SCF:

- Кожен SCF має свою похибку збіжності
- Критерій збіжності: 10^{-8} Ha

Методологічні обмеження

1. Гармонічне наближення:

- Використовується квадратична апроксимація енергії
- Анггармонізм не враховується
- Для високочастотних мод похибка може бути > 100 см⁻¹

2. Якість базису:

- 6-31G — мінімальний базис для якісних результатів
- Для точних інтенсивностей потрібні 6-311++G** або aug-cc-pVTZ

3. Рівень теорії:

- RHF не враховує електронну кореляцію
- DFT (B3LYP) дає кращі результати
- MP2/CCSD — найточніші, але дорогі

Покращення точності**1. Використовувати кращі базиси:**

```
basis = "6-311++G**" # або "aug-cc-pVTZ"
```

2. Використовувати DFT:

```
mf = dft.RKS(mol)
mf.xc = "B3LYP"
```

3. Масштабувати частоти:

Емпіричне масштабування для RHF/6-31G:

$$\nu_{\text{corr}} = 0.8929 \times \nu_{\text{calc}} \quad (1.26)$$

4. Враховувати ангармонізм:

Використовувати VPT2 (Vibrational Perturbation Theory)

Висновок Програма `h2o_ir_spectrum.py` надає повний розрахунок ІЧ-спектру, використовуючи:

- Аналітичний Гесіан для частот (швидко та точно)
- Чисельне диференціювання дипольного моменту для інтенсивностей
- Метод центральних різниць ($O(h^2)$ точність)

Результати є якісно правильними для розуміння коливальної структури молекули. Для кількісних порівнянь з експериментом потрібні:

- Кращі базисні набори
- Врахування електронної кореляції (DFT або пост-HF)
- Ангармонічні корекції
- Врахування ефектів середовища (розчинник)

1.1.7. Раманівський спектр

На відміну від ІЧ, активність у Раман-спектрі визначається поляризованістю α , тобто інтенсивність Раман розсіювання пропорційна зміні поляризованості:

$$I_i^{\text{Раман}} \propto \left| \frac{d\alpha_{ij}}{dQ_k} \right|^2$$

Інтенсивності Раман розсіювання

Правило відбору Раман:

- Раман-активна: $\frac{d\alpha}{dQ_i} \neq 0$.
- Часто доповнює ІЧ (правило взаємного виключення для центросиметричних).
- Для H_2O : всі три моди також Раман-активні

h2o_raman_activity.py

```
# =====
# h2o_raman_activity.py
# =====
from pyscf import gto, scf
from pyscf.prop.polarizability import rhf as pol_rhf
import numpy as np

mol = gto.M(
    atom="""
    O  0.0000  0.0000  0.1173
    H  0.0000  0.7572 -0.4692
    H  0.0000 -0.7572 -0.4692
    """,
    basis="6-31g",
    unit="angstrom",
)

print("Розрахунок Раман-активності H2O")
print("=" * 60)

# SCF
mf = scf.RHF(mol)
mf.kernel()

# Поляризованість
polar = pol_rhf.Polarizability(mf)
alpha = polar.polarizability()

print("\nСтатична поляризованість (au³):")
print(f"αxx = {alpha[0, 0]:.4f}")
print(f"αyy = {alpha[1, 1]:.4f}")
print(f"αzz = {alpha[2, 2]:.4f}")

alpha_mean = np.trace(alpha) / 3
print(f"\nСередня поляризованість: {alpha_mean:.4f} au³")
print(f"Середня поляризованість: {alpha_mean * 0.1482:.4f} Å³")

print("\nРаман-активні моди (якісно):")
print("- Симетричне розтягування OH: сильна")
print("- Деформація HON: середня")
print("- Асиметричне розтягування OH: слабка")
```

Типові активності для H_2O :

Мода	ν (см ⁻¹)	Раман-активність (Å ⁴ /amu)
ν_1	3657	1.8
ν_2	1595	0.4
ν_3	3756	3.2

Принцип взаємного виключення:

Для молекул з центром інверсії (наприклад, CO_2):

- Парні моди (g): Раман активні, ІЧ неактивні
- Непарні моди (u): ІЧ активні, Раман неактивні

1.2. Термохімія та нульова енергія

Розглянемо, як із результатів квантово-хімічного розрахунку (зокрема з гесіяна) отримують термохімічні параметри: нульову коливальну енергію, термічні поправки, ентропію та вільну енергію. Ці параметри *відносяться до газової фази* та розраховуються в рамках *моделі ідеального газу*.

У квантово-хімічних програмах, таких як PySCF, припускається, що молекула є ізольованою частинкою у газовій фазі, а її внутрішня енергія складається з кількох незалежних внесків:

$$E_{\text{total}} = E_{\text{elec}} + E_{\text{vib}} + E_{\text{rot}} + E_{\text{trans}}.$$

Тут E_{elec} — *електронна енергія системи*, отримана з розв'язку електронного рівняння Шредінгера при фіксованих положеннях ядер (наближення Борна–Оппенгеймера). Залежно від рівня теорії, цей внесок може бути визначений у межах методу Хартрі–Фока (HF), теорії функціонала густини (DFT) або більш точних пост-ХФ підходів

Для оцінки цих ядерних внесків використовується *наближення жорсткого ротатора* (для обертання молекули) та *гармонічного осцилятора* (для коливань атомів). Відповідно, термохімічні функції молекул — енергія, ентальпія, ентропія та вільна енергія Гіббса — залежать не лише від електронної будови, але й від термічних ефектів, пов'язаних із поступальним, обертальним і коливальним рухом ядер.

1.2.1. Нульова коливальна енергія (ZPE)

Навіть при абсолютному нулі температури ($T = 0$ К) молекула не є нерухомою. Згідно з принципом невизначеності Гайзенберга, ядра не можуть одночасно мати точно визначені координати й імпульси, тому вони завжди здійснюють квантові коливання навколо рівноважного положення.

Для гармонічного осцилятора енергії рівнів задаються:

$$E_v = \left(n + \frac{1}{2}\right) \hbar \omega, \quad n = 0, 1, 2, \dots$$

Тобто навіть у найнижчому стані ($n = 0$) коливальна енергія дорівнює $\frac{1}{2} \hbar \omega$. Для багатомодової молекули (з $3N - 6$ нормальних мод) нульова коливальна енергія (Zero Point Energy, ZPE) становить:

$$E_{\text{ZPE}} = \sum_{i=1}^{3N-6} \frac{1}{2} \hbar \omega_i.$$

Знання ZPE необхідне для точних термодімічних розрахунків (зокрема енергій реакцій), також вона визначає *ізотопні ефекти*, впливає на швидкість тунелювання через потенційні бар'єри. Для невеликих органічних молекул становить зазвичай 5–15 ккал/моль.

1.2.2. Термодімічні поправки

При підвищенні температури ядра можуть займати збуджені коливальні, обертальні й поступальні стани. Це змінює середню енергію системи, її ентальпію, ентропію та вільну енергію Гіббса.

Для кожного ступеня вільності маємо:

$$E_{\text{vib}}(T) = \sum_i \frac{\hbar\omega_i}{\exp(\hbar\omega_i/k_B T) - 1},$$

$$E_{\text{rot}}(T) = \begin{cases} k_B T, & \text{для лінійної молекули;} \\ \frac{3}{2} k_B T, & \text{для нелінійної.} \end{cases}$$

$$E_{\text{trans}}(T) = \frac{3}{2} k_B T.$$

Тоді повна енергія системи:

$$E_{\text{total}}(T) = E_{\text{elec}} + E_{\text{ZPE}} + E_{\text{vib}}(T) + E_{\text{rot}}(T) + E_{\text{trans}}(T).$$

Ентальпія визначається як:

$$H(T) = E_{\text{total}}(T) + RT.$$

1.2.3. Ентропія молекули

Ентропія — це міра неупорядкованості системи або кількості доступних мікростанів. Для ідеального газу вона має такі складові:

$$S(T) = S_{\text{trans}} + S_{\text{rot}} + S_{\text{vib}} + S_{\text{elec}}.$$

1. Поступальна ентропія:

$$S_{\text{trans}} = R \left[\ln \left(\frac{(2\pi m k_B T)^{3/2} k_B T}{h^3 P} \right) + \frac{5}{2} \right],$$

де m — маса молекули, P — тиск.

2. Обертальна ентропія:

$$S_{\text{rot}} = R \left[\ln \left(\frac{\sqrt{\pi} T^{3/2}}{\sigma} \left(\frac{8\pi^2 k_B}{h^2} \right)^{3/2} \sqrt{I_A I_B I_C} \right) + \frac{3}{2} \right],$$

де σ — число симетричних перестановок, I_A, I_B, I_C — головні моменти інерції.

3. Коливальна ентропія:

$$S_{\text{vib}} = R \sum_i \left[\frac{\theta_i/T}{\exp(\theta_i/T) - 1} - \ln(1 - e^{-\theta_i/T}) \right],$$

де $\theta_i = h\nu_i/k_B$ — характеристична температура моди.

4. Електронна ентропія: Зазвичай нехтують, бо основний стан є єдиним при кімнатних температурах.

1.2.4. Вільна енергія Гіббса

$$G(T) = H(T) - TS(T)$$

Вона визначає термодинамічну стабільність сполук і напрямки хімічних реакцій.

1.2.5. Як це реалізовано в PySCF

У пакеті **PySCF** повний термохімічний аналіз молекули базується на енергії електронів E_{elec} , до якої послідовно додаються внески нульових коливань, теплові поправки та ентропійні терміни.

У пакеті **PySCF** термохімічні величини обчислюються на основі гармонічних частот, отриманих із гессіана (матриці других похідних енергії за координатами атомів), і додаються до електронної енергії E_{elec} , отриманої з розв'язку рівнянь Хартрі–Фока або DFT.

Далі програма:

1. Визначає нормальні частоти ν_i після масового масштабування гессіана.
2. Обчислює нульову енергію коливань $E_{\text{ZPE}} = \frac{1}{2} \sum h\nu_i$.
3. Розраховує термічні внески $E_{\text{vib}}(T), E_{\text{rot}}(T), E_{\text{trans}}(T)$ за статистично-механічними формулами (як наведено вище).
4. Обчислює ентальпію $H(T)$, ентропію $S(T)$ та вільну енергію $G(T)$.

Всі ці розрахунки виконуються у межах *наближення гармонічного осцилятора та жорсткого ротатора*, що забезпечує добру точність для малих і середніх молекул.

1.2.6. Термохімічний приклад: H₂O

`h2o_thermochemistry.py`

```
# =====
# h2o_thermochemistry.py
# Термохімічний аналіз молекули H2O (PySCF)
# =====

import numpy
from pyscf import gto, hessian
from pyscf.hessian.thermo import *
```

```

from pyscf.data import nist

# -----
# МОЛЕКУЛА ВОДИ
# -----
mol = gto.Mole()
mol.atom = '''
O  0.000000  0.000000  0.000000
H  0.000000  0.757000  0.587000
H  0.000000 -0.757000  0.587000
'''
mol.basis = '6-31g(d)'
mol.build()

mass = mol.atom_mass_list(isotope_avg=True)

# -----
# Хартрі-Фок, гессіан, термодинаміка
# -----
mf = scf.RHF(mol).run(verbose=0)
hess = hessian.RHF(mf).kernel()

# -----
# Гармонічний аналіз коливань:
# з гессіану обчислюються власні частоти (в см-1) та нормальні моди.
# Виділяються поступальні й обертальні ступені свободи (6 для нелінійних систем).
# Результат містить частоти, мас-зважений гессіан і нормальні координати.
# -----
results = harmonic_analysis(mol, hess)
# dump_normal_mode(mol, results) # при потребі

# -----
# Розрахунок термодинамічних параметрів при заданій температурі і тиску.
# Функція thermo():
# - використовує вібраційні частоти (results['freq_au'])
# - враховує поступальні, обертальні та коливальні ступені свободи
# - обчислює нульову коливальну енергію (ZPE),
#   термічні поправки до енергії, ентальпію, ентропію,
#   теплоємності (Cv, Cp) та вільну енергію Гіббса (G)
# -----
results = thermo(mf, results['freq_au'], 298.15, 101325)

# =====
# ТЕРМОХІМІЧНІ ПАРАМЕТРИ
# =====
T = results['temperature'][0]
P = results['pressure'][0]

print("\n" + "="*80)
print(f"{'ТЕРМОХІМІЧНІ ПАРАМЕТРИ':^80}")
print("="*80)
print(f"Температура (T): {T:10.2f} {results['temperature'][1]}")
print(f"Тиск (P): {P:10.2f} {results['pressure'][1]}")
print(f"Ротаційні константи [{results['rot_const'][1]}]: "
      f" {results['rot_const'][0][0]:10.5f} {results['rot_const'][0][1]:10.5f} "
      f" {results['rot_const'][0][2]:10.5f}")
print(f"Симетрійне число: {results['sym_number'][0]:>10d}")
print(f"Нульова коливальна енергія (ZPE): {results['ZPE'][0]:10.5f} Eh"
      f" f = {results['ZPE'][0] * nist.HARTREE2J * nist.AVOGADRO:12.3f} Дж/моль")

# =====
# ТЕРМОДИНАМІЧНА ТА ЕНЕРГЕТИЧНА ІНФОРМАЦІЯ
# =====

```

```

# -----
# ТЕРМОДИНАМІЧНА ТАБЛИЦЯ
# -----
keys = ('tot', 'elec', 'trans', 'rot', 'vib')
header = ["Функція", "Одиниці"] + [x.upper() for x in keys]

def convert(f, keys, unit):
    """Переведення значень термодинамічних функцій у потрібні одиниці"""
    conv = nist.HARTREE2J * nist.AVOGADRO if 'Eh' in unit else 1
    return [results.get(f + '_' + key, (0,))[0] * conv for key in keys]

def write_table_row(title, f):
    """Створює один рядок термодинамічної таблиці"""
    tot, unit = results[f + '_tot']
    values = convert(f, keys, unit)
    # Заміна одиниць для більш зрозумілого відображення
    unit = unit.replace('Eh', 'J/mol·K')
    return [title, unit] + [f"{v:10.3f}" for v in values]

# Формуємо таблицю для S, Cv, Cp
table = [
    write_table_row("Ентропія S", "S"),
    write_table_row("Cv", "Cv"),
    write_table_row("Cp", "Cp"),
]

print("\n" + "="*100)
print(f"{f'ТЕРМОДИНАМІЧНІ ФУНКЦІЇ (T = {T:.2f} K, P = {P/101325:.2f} atm)'}:^100}")
print("="*100)
print(f"{header[0]:<15s} {header[1]:<15s} " + " ".join(f"{h:>10s}" for h in header[2:]))
print("-"*100)
for row in table:
    print(f"{row[0]:<15s} {row[1]:<15s} " + " ".join(row[2:]))
print("-"*100)

# -----
# ЕНЕРГЕТИЧНА ТАБЛИЦЯ (у kJ/mol)
# -----
Ha2kJ = nist.HARTREE2J * nist.AVOGADRO / 1000 # 1 Eh → kJ/mol

def convert_kJ(f, keys):
    """Повертає список енергій у kJ/mol для заданої функції."""
    return [results.get(f + '_' + key, (0,))[0] * Ha2kJ for key in keys]

def write_energy_row_kJ(title, f):
    values = convert_kJ(f, keys)
    return [title, "kJ/mol"] + [f"{v:12.3f}" for v in values]

# --- ZPE ---
ZPE_kJ = results['ZPE'][0] * Ha2kJ
ZPE_row = ["ZPE", "kJ/mol", f"{ZPE_kJ:12.3f}"]

energy_table_kJ = [
    ZPE_row,
    write_energy_row_kJ("E", "E"),
    write_energy_row_kJ("H", "H"),
    write_energy_row_kJ("G", "G"),
]

print("\n" + "="*100)
print(f"{f'ЕНЕРГЕТИЧНІ ВЕЛИЧИНИ (T = {T:.2f} K, P = {P/101325:.2f} atm)'}:^100}")
print("="*100)
print(f"{header[0]:<15s} {header[1]:<15s} " + " ".join(f"{h:>12s}" for h in header[2:]))
print("-"*100)

```

```
for row in energy_table_kJ:
    print(f"{row[0]:<15s} {row[1]:<15s} " + " ".join(row[2:]))
print("="*100)
```

Зауваження до програми h2o_thermochemistry.py

У цій програмі основна робота термохімічного аналізу була зведена до двох ключових функцій з модуля `pysch.hessian.thermo`:

1. `harmonic_analysis(mol, hess)` — виконує гармонічний аналіз молекули на основі гесіану. Вона обчислює частоти нормальних коливань, нормалізовані моди, редуковані маси та нульову коливальну енергію (ZPE). Ця функція також враховує трансляційні та обертальні ступені свободи та дозволяє виключати їх при необхідності.
2. `thermo(model, freq, temperature, pressure)` — використовує результати гармонічного аналізу для обчислення термодинамічних величин: ентропії, теплоємності (C_V , C_P), внутрішньої енергії, ентальпії та вільної енергії Гіббса. Вона враховує внески електронної, поступальної, обертальної та коливальної частин.

Таким чином, за рахунок цих двох функцій програма зводить складні квантово-хімічні розрахунки та гармонічну апроксимацію в зручний формат для термохімічного аналізу. Вся «магія» полягає в тому, що `harmonic_analysis` формує основні фізичні параметри, а `thermo` перетворює їх у термодинамічні величини, готові для подальшого виводу у таблиці.

1.2.7. Перехідні стани та уявні частоти

Класифікація стаціонарних точок

Гесіан — матриця других похідних потенціальної енергії за атомними координатами — дає змогу класифікувати стаціонарні точки потенціальної поверхні енергії (ППЕ) за знаками власних значень:

$$\lambda_i = \frac{\partial^2 E}{\partial q_i^2}.$$

Тип	Уявних частот	Характеристика
Мінімум	0	Локальний мінімум
Перехідний стан	1	Сідлова точка 1-го порядку
Сідло 2-го порядку	2	Нестабільна структура

Уявна частота: якщо $\omega^2 < 0$, то $\omega = i|\omega|$, і відповідний напрямок руху описує нестійку моду.

Інверсія аміаку (NH_3) як приклад

Молекула аміаку має тригранно-пірамідальну структуру симетрії C_{3v} : атом Нітрогену розташований над площиною трьох атомів Гідрогену. Дзеркально-еквівалентна конфігурація виникає, коли атом N переходить на протилежний бік площини — процес, відомий як *парасолькова інверсія*.

Якщо ввести координату h , що задає відстань атома N від площини H_3 , то потенціальна енергія має форму подвійного мінімуму:

$$E(h) \approx E_0 + \frac{1}{2}kh^2 + \frac{1}{4}\lambda h^4, \quad \lambda > 0.$$

Пірамідальні структури відповідають точкам $h = \pm h_0$ (локальні мінімуми), а планарна структура ($h = 0$) є **перехідним станом**.

Аналіз гесіану У рівноважній геометрії ($h = h_0$) усі власні значення гесіану додатні, що відповідає стабільному мінімуму:

$$\lambda_i > 0 \quad \Rightarrow \quad \omega_i = \sqrt{\lambda_i/\mu_i} > 0.$$

Для планарної конфігурації ($h = 0$) одне власне значення стає від'ємним, і відповідна частота набуває уявного значення:

$$\omega = i|\omega|, \quad \omega^2 < 0.$$

Це свідчить про сідлову точку першого порядку — перехідний стан інверсії аміаку.

Квантово-механічна інтерпретація У класичному наближенні атом N мав би подолати потенціальний бар'єр, однак квантова механіка допускає *тунелювання*. Енергетичне розщеплення між ними ($\Delta E \approx 0.79 \text{ см}^{-1}$) відповідає частоті мікрохвильового переходу, що спостерігається експериментально в *амонійному лазері*.

Обчислення в PySCF Профіль потенціальної енергії інверсії можна отримати чисельно, змінюючи координату h :

1. Задати кілька значень h і для кожного провести оптимізацію решти координат.
2. Побудувати криву $E(h)$ та визначити висоту бар'єра.
3. У мінімумі ($h = h_0$) усі частоти дійсні та додатні.
4. У планарній структурі ($h = 0$) з'являється одна уявна частота — ознака перехідного стану.

[nh3_inversion.py](#)

```
import numpy as np
from pyscf import gto, scf
from pyscf.hessian import thermo
```

```

from scipy.optimize import minimize_scalar
import matplotlib.pyplot as plt

# =====
# ЕТАП 1: ГРУБИЙ ПОШУК МАКСИМУМУ ЕНЕРГІЇ
# =====
def energy_func(h):
    """Енергія системи при висоті N = h над площиною NH3"""
    mol = gto.M(
        atom=f'''
        N   0.0000   0.0000   {h}
        H   0.0000   1.0124   0.0000
        H   0.8770  -0.5062   0.0000
        H  -0.8770  -0.5062   0.0000
        ''',
        basis='6-31g**',
        unit='angstrom'
    )
    mf = scf.RHF(mol)
    mf.verbose = 0
    return mf.kernel()

print("=" * 70)
print("АНАЛІЗ ІНВЕРСІЇ NH3")
print("=" * 70)

# =====
# ЕТАП 2: ДЕТАЛЬНИЙ АНАЛІЗ ІЗ ГЕССИАНОМ І ЧАСТОТАМИ
# =====
def analyze_point(h, verbose=True):
    """Повний аналіз точки: енергія, градієнт, гессіан, частоти"""
    mol = gto.M(
        atom=f'''
        N   0.0000   0.0000   {h}
        H   0.0000   1.0124   0.0000
        H   0.8770  -0.5062   0.0000
        H  -0.8770  -0.5062   0.0000
        ''',
        basis='6-31g**',
        unit='angstrom',
        symmetry=False
    )
    mf = scf.RHF(mol)
    mf.verbose = 0
    e = mf.kernel()

    # Градієнт
    grad = mf.Gradients().kernel()
    grad_signed = grad[0, 2] # градієнт по z атома N

    # Гессіан і частоти
    if verbose:
        print(f"  Вичислення гессіана для h = {h:.4f} Å...")
    hess = mf.Hessian().kernel()
    freq_info = thermo.harmonic_analysis(mol, hess, exclude_trans=True, imaginary_freq=True)
    frequencies = freq_info['freq_wavenumber']

    # Підсчёт мнмих частот
    imag_freqs = [f for f in frequencies if np.iscomplex(f)]
    num_imag = len(imag_freqs)

    return {

```

```

        'h': h,
        'energy': e,
        'grad_signed': grad_signed,
        'num_imag': num_imag,
        'imag_freqs': imag_freqs,
        'all_freqs': frequencies
    }

# Груба сітка по всьому діапазону
h_coarse = np.linspace(-0.25, 0.25, 10)

# Уточнення поблизу мінімумів (~ ±0.215 Å)
h_fine_minima = np.linspace(-0.23, -0.20, 15)
h_fine_minima2 = np.linspace(0.20, 0.23, 15)

# Уточнення поблизу перехідного стану (h ≈ 0)
h_fine_ts = np.linspace(-0.02, 0.02, 15)

# Об'єднуємо все разом
extra_points = [0.0, -0.215, 0.215]
h_detailed = np.unique(np.sort(np.concatenate([
    h_coarse, h_fine_minima, h_fine_minima2, h_fine_ts, extra_points
])))

results = []

print(f"\n{'h' (Å)':<10} {'E_rel (ккал/моль)':<18} {'grad E':<12} {'Частоти (см⁻¹)':<50}")
print("-" * 100)

grad_tol = 1e-4      # поріг малості градієнта
imag_tol = 1e-6      # поріг для "нульової" уявної частини

results = []

for h in h_detailed:
    res_point = analyze_point(h, verbose=False)
    results.append(res_point)

    # енергія відносно мінімуму (за всіма вже порахованими)
    e_rel = (res_point['energy'] - min(r['energy'] for r in results)) * 627.51

    grad_signed = res_point['grad_signed']

    # показуємо частоти, якщо градієнт малий
    show_freq = abs(grad_signed) < grad_tol

    if show_freq:
        freqs_raw = np.array(res_point['all_freqs'], dtype=complex)
        freq_items = []
        for f in freqs_raw:
            if np.iscomplexobj(f) and abs(f.imag) > imag_tol:
                val = abs(f.imag) if abs(f.real) < imag_tol else np.sqrt(f.real**2 + f.imag**2)
                freq_items.append(f"i{val:.1f}")
            else:
                fr = float(np.real(f))
                if fr < 0:
                    freq_items.append(f"i{abs(fr):.1f}")
                else:
                    freq_items.append(f"{fr:.1f}")
        freq_display = ", ".join(freq_items)
    else:
        freq_display = ""

    print(f"h:<10.4f} {e_rel:<18.4f} {grad_signed:<12.2e} {freq_display:<60}")

```



```

# =====
# АНАЛІЗ ПЕРЕХІДНОГО СТАНУ
# =====
print("\n" + "=" * 70)
print("АНАЛІЗ ПЕРЕХІДНОГО СТАНУ")
print("=" * 70)

# Шукаємо точки з максимальною енергією та мінімальною енергією
energies = [r['energy'] for r in results]
max_idx = np.argmax(energies)
min_idx = np.argmin(energies)
e_min = energies[min_idx]
ts_candidate = results[max_idx]

print(f"\nКандидат на TS: h = {ts_candidate['h']:.6f} Å")
print(f"Відносна енергія: {(ts_candidate['energy'] - e_min) * 627.51:.4f} ккал/моль")
print(f"Норма градієнта: {ts_candidate['grad_signed']:.2e}")
print(f"Число уявних частот: {ts_candidate['num_imag']}")

# Перевірка критеріїв TS
is_ts = True
issues = []

if abs(ts_candidate['h']) > 0.02:
    is_ts = False
    issues.append(f"h не близьке до 0 (h = {ts_candidate['h']:.4f} Å)")

if ts_candidate['grad_signed'] > 5e-3:
    is_ts = False
    issues.append(f"Гرادієнт занадто великий ({ts_candidate['grad_signed']:.2e})")

if ts_candidate['num_imag'] != 1:
    is_ts = False
    issues.append(f"Не 1 уявна частота (знайдено {ts_candidate['num_imag']})")

if is_ts:
    print("\n✓✓✓ ЦЕ ПЕРЕХІДНИЙ СТАН! ✓✓✓")
    imag_freq = abs(ts_candidate['imag_freqs'][0])
    print(f"\nУявна частота інверсії: i{imag_freq:.2f} см-1")
    print(f"Бар'єр інверсії: {(ts_candidate['energy'] - e_min) * 627.51:.4f} ккал/моль")
else:
    print("\n❌ НЕ ПЕРЕХІДНИЙ СТАН")
    print("\nПроблеми:")
    for issue in issues:
        print(f"• {issue}")

# =====
# ВІЗУАЛІЗАЦІЯ
# =====
fig, ax1 = plt.subplots(1, 1, figsize=(14, 5))

# Графік профілю енергії
h_vals = [r['h'] for r in results]
e_vals = [(r['energy'] - e_min) * 627.51 for r in results]
ax1.plot(h_vals, e_vals, 'r-', linewidth=2, label='Профіль енергії')

ax1.axvline(ts_candidate['h'], color='orange', linestyle=':', linewidth=2,
            label=f"Максимальна енергія: h={ts_candidate['h']:.3f} Å")
ax1.set_xlabel('Висота N над площиною H3 (Å)', fontsize=11)
ax1.set_ylabel('Відносна енергія (ккал/моль)', fontsize=11)
ax1.set_title('Профіль енергії інверсії NH3', fontsize=12, fontweight='bold')

```

```

ax1.legend(fontsize=9)
ax1.grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig('nh3_inversion.pdf', dpi=300, bbox_inches='tight')
print(f"\n✓ Графік збережено: nh3_inversion_ts_full.pdf")

# =====
# ПІДСУМКОВЕ ЗВЕДЕННЯ
# =====

print(f"\nПерехідний стан (з аналізу):")
print(f"  h(N) = {ts_candidate['h']:.6f} Å")
print(f"  Бар'єр = {(ts_candidate['energy'] - e_min) * 627.51:.4f} ккал/моль")
if ts_candidate['num_imag'] > 0:
    print(f"  Уявна частота = i{abs(ts_candidate['imag_freqs'])[0]:.2f} см-1")
print("  " * 70)

```

Фізичний зміст У планарній конфігурації ($h = 0$) гесіан має одне від'ємне власне значення, що відповідає уявній частоті

$$\omega = i 685.7 \text{ см}^{-1}.$$

Власний вектор, що відповідає цій частоті, описує *парасолькову моду* — рух атома Нітрогену перпендикулярно до площини H_3 зі зустрічним рухом атомів Гідрогену. Ця мода визначає **реакційну координату** інверсії: перехід між двома пірамідальними мінімумами через планарний бар'єр.

Невелика висота бар'єра ($\Delta E \approx 1.08$ ккал/моль) забезпечує ефективне тунелювання, через що спостерігається розщеплення основного коливального рівня. У спектрах це проявляється як подвоєння мікрохвильового сигналу або розщеплення в ЯМР через різні середні положення ядер.

Таким чином, уявна частота в гесіані має чіткий фізичний сенс: вона вказує напрямок реакційного руху, що веде з перехідного стану до двох еквівалентних рівноважних конфігурацій, і описує квантову природу інверсії молекули аміаку.